# Newton Raphson Method

SanjeevK(ph25mscst11026)

## Question 1

1. Consider the function $f(x) = 2\cos x - x$. Consider the three initial guesses $x_0 = 0.5$, $\pi$ & $2\pi$ and for these guesses find the roots using Newton-Raphson method.

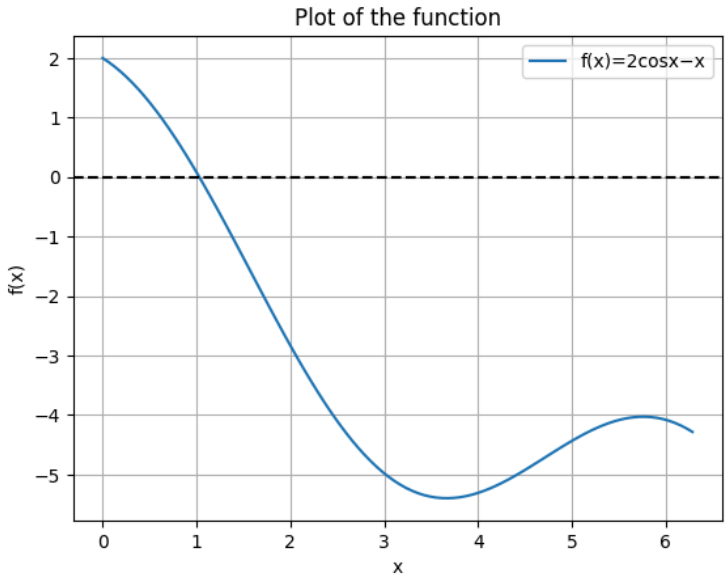
```
#imports
import numpy as np
import matplotlib.pyplot as plt
```

```
#defining the function
def f(x):
  return 2 * np.cos(x) - x

#derivative of the function
def df(x):
  return -2 * np.sin(x) - 1


x_0 = 0.5  #initial guess
N_max = 100   #maximum number of iteration
tol = 1e-6
roots = []


for i in range(N_max):
    x = x_0 - f(x_0)/df(x_0)
    roots.append(x)

    if abs(x - x_0) < tol and abs(f(x)) < tol:
        break

    x_0 = x

roots = np.array(roots, dtype=float)

#print the results
print("Root =", roots[-1])
print("No of Iterations =", i+1)
print("Iterations:", roots)
```

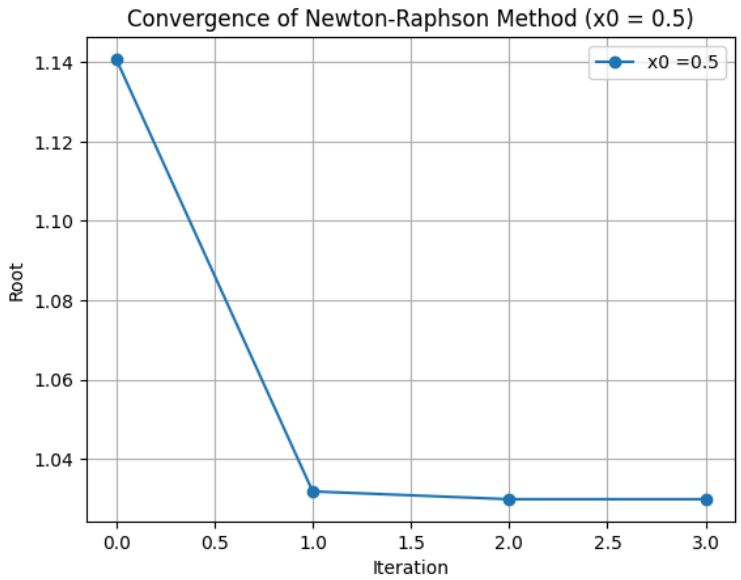
```
Root = 1.02986652932236
No of Iterations = 4
Iterations: [1.14076598 1.03183077 1.02986726 1.02986653]
```

```
#Graphs
x = np.linspace(0, 2 * np.pi, 100)
```

```
#plot of the function
plt.plot(x, f(x), label='f(x)=2cosx-x')
plt.axhline(0, color='black', linestyle='--')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Plot of the function')
plt.grid()
plt.legend()
plt.show()
```




```
#convergence of the newton raphson method
plt.plot(range(len(roots)), roots, marker='o', label='x0 =0.5')
plt.xlabel('Iteration')
plt.ylabel('Root')
plt.title('Convergence of Newton-Raphson Method (x0 = 0.5)')
plt.grid()
plt.legend()
plt.show()
```



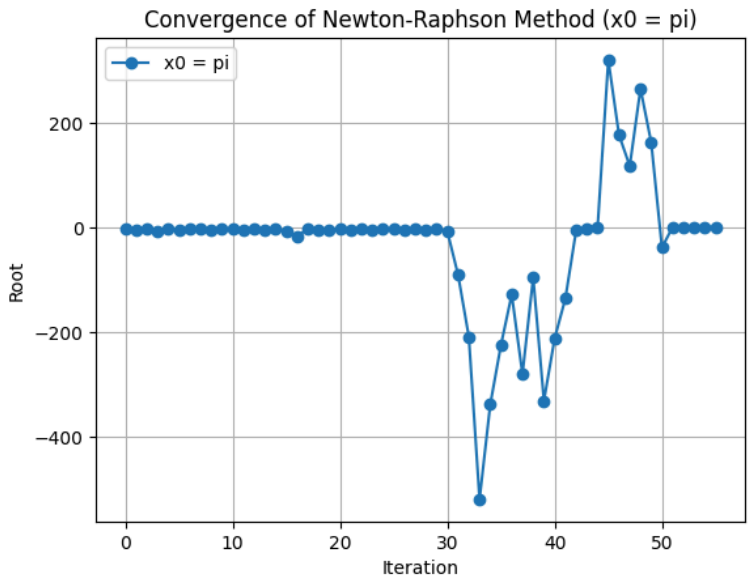

for initial guess $x_0 = \pi$

```
x_0 = np.pi

N_max = 100   #maximum number of iteration
tol = 1e-6
roots = []


for i in range(N_max):
    x = x_0 - f(x_0)/df(x_0)
    roots.append(x)

    if abs(x - x_0) < tol and abs(f(x)) < tol:
        break

    x_0 = x

roots = np.array(roots, dtype=float)

#print the results
print("Root =", roots[-1])
print("No of Iterations =", i+1)
print("Iterations:", roots)
```

```
Root = 1.0298665293222589
No of Iterations = 56
Iterations: [  -2.           -3.42647651   -2.46168831   -5.98263353   -1.02502602
   -3.93314304   -2.88990147   -0.99134274   -4.08919302   -2.97566871
   -1.47771849   -3.15585197   -2.03185056   -3.47536903   -2.517346
   -7.81134033  -15.72231963   -2.38279482   -4.85944877   -3.12949713
   -1.97184968   -3.38760173   -2.41399773   -5.20195072   -2.98052012
   -1.49886503   -3.15000814   -2.01896551   -3.45492334   -2.49461436
```

```
  -6.86712437  -90.03767712 -208.33593111 -519.55211507 -337.91191212
 -223.7687419  -127.64560667 -279.85632779  -95.02937401 -330.94740232
 -211.55681545 -135.10140755   -5.3533231    -2.83737113   -0.51955433
  321.10113544  177.44226454  118.26760388  266.1087828   163.26392084
  -37.69317648    1.5343086     1.04697591    1.02992066    1.02986653
    1.02986653]
```

```
#convergence of the newton raphson method
plt.plot(range(len(roots)), roots, marker='o', label='x0 = pi')
plt.xlabel('Iteration')
plt.ylabel('Root')
plt.title('Convergence of Newton-Raphson Method (x0 = pi)')
plt.legend()
plt.grid()
plt.show()
```



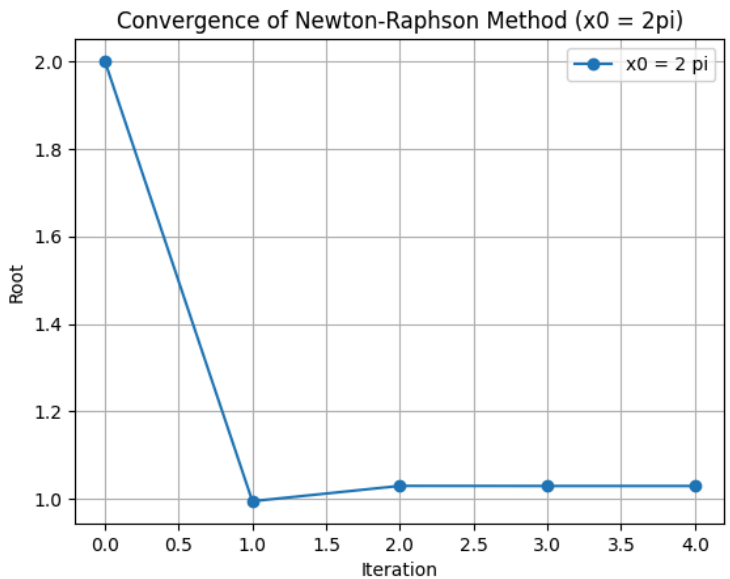

for initial guess $x_0 = 2\pi$

```
x_0 = 2 * np.pi

N_max = 100   #maximum number of iteration
tol = 1e-6
roots = []


for i in range(N_max):
    x = x_0 - f(x_0)/df(x_0)
    roots.append(x)

    if abs(x - x_0) < tol and abs(f(x)) < tol:
        break

    x_0 = x
```

```
roots = np.array(roots, dtype=float)

#print the results
print("Root =", roots[-1])
print("No of Iterations =", i+1)
print("Iterations:", roots)
```

```
Root = 1.0298665293222589
No of Iterations = 5
Iterations: [2.         0.99513984 1.03010731 1.02986654 1.02986653]
```

```
#convergence of the newton raphson method
plt.plot(range(len(roots)), roots, marker='o', label='x0 = 2 pi')
plt.xlabel('Iteration')
plt.ylabel('Root')
plt.title('Convergence of Newton-Raphson Method (x0 = 2pi)')
plt.legend()
plt.grid()
plt.show()
```




## Question 2

2. Compare the convergence against bisection method where the ranges are $[0.5, -2]$ , $[0, \pi]$ and $[0, 2\pi]$. Briefly report your observation for both the root finding methods.

```
#calculating roots using bisection method

#Define Function
def f(x):
  return 2 * np.cos(x) - x

print(f"The value of function at x = 0.5 is : {f(0.5)}")
```

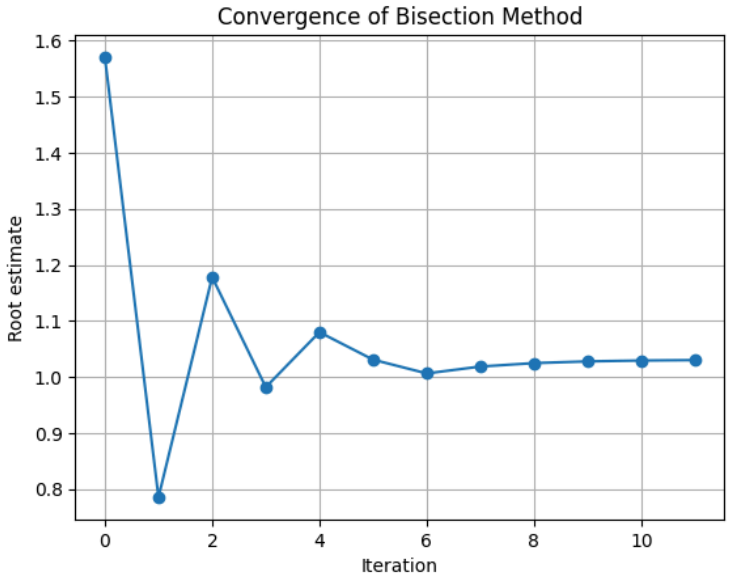
```
print(f"The value of function at x = -2 is : {f(-2)}")
print(f"The value of function at x = 0 is : {f(0)}")
print(f"The value of function at x = pi is : {f(np.pi)}")
print(f"The value of function at x = 2pi is : {f(2 * np.pi)}")
print(f"Is the product of the function at x=0.5 and -2 less than zero: {f(0.5) * f(-2) < 0 }")
```

```
The value of function at x = 0.5 is : 1.2551651237807455
The value of function at x = -2 is : 1.1677063269057153
The value of function at x = 0 is : 2.0
The value of function at x = pi is : -5.141592653589793
The value of function at x = 2pi is : -4.283185307179586
Is the product of the function at x=0.5 and -2 less than zero: False
```

we can see that the value of the function at x = 0 and x = -2 are positive so $f(-2)f(0.5) > 0$ so root does not lies between this range. we can't calculate root for this range using bisection method. for the other given two ranges $f(a)f(b) < 0$ so we can find roots using bisection method

```
#Take values of x in the range[0, pi]

#finding the roots of f(x) = 0 using bisection method
eps = 1e-3
Nmax = 100  #Maximum number of iterations
a = 0
b = np.pi

#Define Bisection Method
def Bisection(Xminus, Xplus, Nmax, eps):
    roots = []

    for it in range(Nmax):
        x = (Xplus + Xminus) / 2.0
        roots.append(x)

        if f(Xplus) * f(x) > 0.0:
            Xplus = x
        else:
            Xminus = x

        if abs(f(x)) < eps:
            print("Converged in", it+1, "iterations")
            break

    return np.array(roots)
roots = Bisection(a, b, Nmax, eps)

plt.figure()
plt.plot(range(len(roots)), roots, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Root estimate")
plt.title("Convergence of Bisection Method")
plt.grid(True)
plt.show()

print("Root =", roots[-1])
print("Iterations:", roots)
```

```
Converged in 12 iterations
```

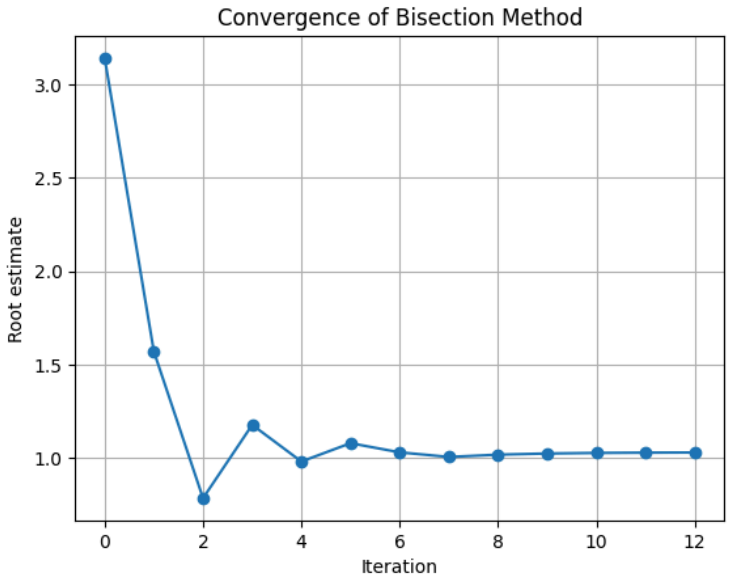



```
Root = 1.030068099065208
Iterations: [1.57079633 0.78539816 1.17809725 0.9817477  1.07992247 1.03083509
 1.0062914  1.01856324 1.02469917 1.02776713 1.02930111 1.0300681 ]
```

```
#Take values of x in the range[0, 2pi]

#finding the roots of f(x) = 0 using bisection method
eps = 1e-3
Nmax = 100  #Maximum number of iterations
a = 0
b = 2 * np.pi

#Define Bisection Method
def Bisection(Xminus, Xplus, Nmax, eps):
    roots = []

    for it in range(Nmax):
        x = (Xplus + Xminus) / 2.0
        roots.append(x)

        if f(Xplus) * f(x) > 0.0:
            Xplus = x
        else:
            Xminus = x

        if abs(f(x)) < eps:
            print("Converged in", it+1, "iterations")
            break

    return np.array(roots)
roots = Bisection(a, b, Nmax, eps)

plt.figure()
plt.plot(range(len(roots)), roots, marker='o')
plt.xlabel("Iteration")
```

```
plt.ylabel("Root estimate")
plt.title("Convergence of Bisection Method")
plt.grid(True)
plt.show()

print("Root =", roots[-1])
print("Iterations:", roots)
```

```
Converged in 13 iterations
```




```
Root = 1.030068099065208
Iterations: [3.14159265 1.57079633 0.78539816 1.17809725 0.9817477  1.07992247
 1.03083509 1.0062914  1.01856324 1.02469917 1.02776713 1.02930111
 1.0300681 ]
```

## Conclusion

The root of $f(x) = 2cosx - x$ was computed using both Newton–Raphson and Bisection methods. Newton–Raphson converged much faster and required fewer iterations when the initial guess was close to the root, showing quadratic convergence. However, its performance depended strongly on the starting value. The Bisection method converged more slowly but provided stable and guaranteed convergence when the root was properly bracketed. Thus, Newton–Raphson is faster but sensitive, while Bisection is slower but more reliable.

```
#I really enjoyed doing this assignment:)
```